

Project Journey

Project Journey — Every Phase, Every Problem, Every Real Fix

This is a chronological record of the work done on Baseera: what phase it belonged to, what concrete problem was found, how it was diagnosed, and — critically — what was **actually applied and verified**, not just proposed. Every claim below is backed by a test run, a real log, a real API response, or a committed file; where a fix did **not** fully work, that is stated honestly rather than smoothed over.

Phase 1 — Results integrity (technical review issues #01–#05)

#	Problem found	Root cause	Fix applied	Verified by
1	<code>analysis_id:</code> <code>null</code> on every live <code>/predict</code> call despite the database showing “connected”	Render has no separate migration/release step — <code>DATABASE_URL</code> connected fine, but the schema had zero tables, so every write silently failed inside the best-effort persistence layer	Alembic migrations now run automatically at app startup (<code>_run_pending_migrations()</code> in <code>main.py</code>)	Live curl to <code>/predict</code> returning a real <code>analysis_id</code>
2	A retraining run once overwrote a model checkpoint without regenerating the metrics file describing it — a silent, undetectable drift	No mechanism compared the published metrics’ recorded checkpoint hash against the checkpoint actually on disk	<code>checkpoint_fingerprint()</code> (SHA-256 over weight files) + <code>_check_metrics_freshness()</code> logs an error at startup on mismatch	<code>scripts/verify_metrics_freshness.py</code> added to CI
3	CORS allowed <code>methods=</code> <code>["*"]</code> , <code>headers=</code> <code>["*"]</code> — far broader than the API’s actual surface	Copy-pasted default, never tightened	Explicit <code>["GET", "POST"]</code> / <code>["Content-Type", "X-API-Key", "Idempotency-Key"]</code>	Manual review; live CORS behavior confirmed correct (see Production Incident 2 below for the <i>other</i> CORS bug this didn’t cause)
4	<code>README.md</code> and the FastAPI <code>description</code> field both claimed “end-to-end” / implied a live data source	The dashboard is a static Olist snapshot (2016–2018), not live/streaming	Reworded both to state this explicitly	Manual review
5	Historical notebook-	—	<code>/models/info</code> now separates <code>metrics</code> (from the corrected	API response inspected directly

#	Problem found	Root cause	Fix applied	Verified by
	reported metrics were presented without flagging they came from a leaky split		pipeline) from <code>historical_notebook_metrics</code> with <code>valid: false</code>	

Phase 2 — Security & availability (#10, #11, #13, #14, #15)

Problem	Failure scenario it closes	Fix
Rate limiter keyed on the raw socket peer address	Behind Render's reverse proxy, that's <i>always</i> the proxy's own IP — every real client shared one shared limit bucket, meaning one heavy user could throttle everyone	<code>client_identity()</code> parses <code>X-Forwarded-For</code> at a configured <code>TRUSTED_PROXY_HOPS</code> position (1 = trust exactly one hop, matching Render/Vercel)
Unbounded batch-upload file reads	A large/malicious upload could buffer an entire file in memory before any row-count cap kicked in — real OOM risk on a 512MB host	Bounded chunked reads, hard 5MB cap, rejects before fully buffering
<code>upload_id</code> read straight into a filesystem path	Path traversal if a crafted ID were ever accepted	Strict 32-hex regex + resolved-path containment check before any file I/O
No concurrency/timeout control on batch classification	Two uploads landing seconds apart could each hold a full batch's model activations in memory at once	<code>asyncio.Semaphore(2)</code> + <code>asyncio.wait_for(..., timeout=300)</code> , degrading to a clean 504 instead of hanging or OOM-ing

Phase 3 — ML correctness (#06–#09)

RFM train/serve skew. The scaler was fit on raw values, then log-transformed separately at serving time — not the same transform the model trained on. Fixed by wrapping `log1p` + `StandardScaler` in one `sklearn.Pipeline`, pickled and reused identically at both train and serve time. Verified by `test_rfm_pipeline_transform_is_identical_at_train_and_serve`.

ABSA hallucination. The aspect model was forced to score all 5 fixed aspects on every review regardless of content ("product quality: Positive 75%" on a delivery-only review). Fixed with a keyword-presence gate; unmentioned aspects now report "Not mentioned". Then **generalized further**: the keyword lists only worked for Olist's exact domain — replaced with a RAKE-based, domain-general extraction pipeline (`app/ml/aspect_extraction.py`), empirically validated on non-e-commerce example reviews too (a

semantic-similarity layer was *also* tried to catch aspects discussed without their own name, measured unreliable, and **not shipped** — an honest negative result, not hidden).

Fake-review instability — first discovered here. Testing `jb10231/fake-review-detector` directly found predictions flipped from 99.9% to 0.1% confidence under a pure meaning-preserving synonym substitution. This became the seed of the multi-week investigation documented in full below.

Phase 4 — Engineering discipline (#17, #18, #20)

- **requirements.txt had no upper bounds.** A fresh install could silently resolve a breaking dependency months later with zero warning. Pinned every dependency with a documented reason per line — and this was **not theoretical caution**: it caught a real bug live in CI (see below).
- **Dockerfile ran as root, had no HEALTHCHECK, no --proxy-headers.** Without `--proxy-headers`, uvicorn ignores `X-Forwarded-For` entirely, silently breaking the Phase 2 rate-limiting fix in production specifically (it would work locally, fail live). Fixed incrementally, then fully hardened into a genuine multi-stage build (see “Docker hardening” below).
- **Dev-only tooling (matplotlib, pytest, datasets) was mixed into the production requirements.txt.** Split into `requirements.txt` (runtime only) + `requirements-dev.txt` (adds test/training tooling) — the deployed image no longer pays for dependencies it never imports.

Phase 5 — Test coverage & doc accuracy (#12, #16, #21, #22)

- **Idempotency key:** `POST /predict` reads an `Idempotency-Key` header; a client retry after a timeout now replays the saved result instead of creating a duplicate history row. Required a new DB migration (`b1c8891786a0`), a unique constraint, and a repository lookup method — all added and tested.
- **Upload concurrency limit** (see Phase 2, formalized and tested here).
- Filled real gaps in test coverage the review flagged, and corrected doc claims that had drifted from what the code actually did.

CI/CD — set up, then immediately caught two real bugs

A GitHub Actions workflow (`backend` / `frontend` / `docker` jobs) was added. **The very first real run failed on both applicable jobs** — and both failures were genuine, not CI flakiness:

1. **Frontend:** `tsc -b` (the build’s actual type-check mode) didn’t honor `/// <reference types="vitest/config" />` the way `tsc --noEmit` had — fixed by importing `defineConfig` directly from `"vitest/config"`. That fix then exposed a **second**, previously-masked bug: `vitest@2.1.8` pulled in a duplicate, nested Vite 5 install that conflicted with the project’s Vite 6, causing a type mismatch. Fixed by upgrading to `vitest@^3.0.5`.
2. **Backend:** a fresh install resolved `fastapi==0.119.1` + `pydantic==2.13.4` — a real, confirmed combination bug (`UploadFile` `TypeAdapter/forward-ref` error) that cascaded into ~26 unrelated test failures via a shared schema-build failure. Diagnosed by comparing CI’s freshly-resolved versions against

the main dev environment's already-working `fastapi==0.141.1`. Fixed by raising the pinned upper bound to `<0.145`, with the exact bug documented inline in `requirements.txt` so it's not silently reintroduced later.

Verified: 129 backend tests + 9 frontend tests passing before either fix was pushed.

Docker hardening — verified before touching production

The Dockerfile was rewritten as a genuine multi-stage build: a `builder` stage installs dependencies (needs the full pip/wheel toolchain), the final stage copies only the installed packages + source — plus switching to PyTorch's official CPU-only wheel (`--extra-index-url .../whl/cpu`) instead of the ~2GB+ CUDA-bundled default, since this app's `device` always resolves to `"cpu"` in production.

This was **not pushed straight to main**: it was built on a separate branch (`harden-dockerfile`), opened as a PR, and only merged after the CI `docker` job — a genuine `docker build` + container boot + real `curl` health check — passed. Confirmed green (`backend` , `frontend` , `docker` all `success`) before merge.

Production incidents — found by checking the live site, not assumed from a green push

Incident 1 — every deploy since a specific commit had been silently failing (OOM)

After merging the Dockerfile PR, Render's deploy for that exact commit showed **"Deploy failed"** with notice: *"Ran out of memory (used over 512MB)."* Investigating further revealed the live site had actually been running a **stale build from several commits earlier** — every deploy in between (all of Phases 1–5) had been failing the same way, and nobody had noticed because the *old* process kept answering `/health` as "healthy" the whole time.

Root cause, found by reading the code, not guessing: `_check_metrics_freshness()` called `checkpoint_fingerprint()`, which did `path.read_bytes()` — loading the *entire* ~670MB BERT checkpoint into memory in one shot, on every single startup, regardless of whether `ENABLE_BERT` was even true.

Fix: hash in 1MB chunks instead of one giant read (verified: identical SHA-256 output to the old method), and skip the check entirely when `ENABLE_BERT=false` — no reason to fingerprint a model that's never loaded. Pushed, redeploy confirmed **"Deploy succeeded | Live"**.

Incident 2 — CORS silently broke the entire public dashboard

Even with the OOM fix live and the backend reporting 100% healthy, the actual deployed frontend showed `NETWORK_ERROR` on every single chart. Reading the browser's own console (not just checking the backend) showed the real error: `Access to XMLHttpRequest ... blocked by CORS policy: No 'Access-Control-Allow-Origin' header`. `FRONTEND_ORIGINS` on Render still held a stale/placeholder value, not the actual deployed Vercel URL.

Fix: updated the `FRONTEND_ORIGINS` environment variable on Render to the real Vercel URL. Re-verified directly in the browser: zero console errors, real dashboard numbers loading (99,441 orders, 96,096 customers, etc.).

Incident 3 — a version-drift risk caught before it caused a third incident

Live logs showed `InconsistentVersionWarning: Trying to unpickle estimator ... from version 1.9.0 when using version 1.5.2` — the RFM scaler/K-Means artifacts had been pickled with a locally-installed scikit-learn newer than what `requirements.txt` pins for production. This is exactly the failure mode the requirements file's own upper-bound comment already warned about. Flagged for re-pickling with a production-matching scikit-learn version (the same discipline later applied when building the fake-review TF-IDF artifact, using an isolated venv pinned to `scikit-learn==1.5.2` specifically to avoid repeating this exact mistake).

The fake-review detector — the longest single investigation in this project

Full technical detail: `MODEL_COMPARISON_AUDIT.md` §9. Summary of the actual sequence:

1. **Checkpoint #1** (`jb10231/fake-review-detector` , external): confirmed unreliable — label semantics never verified against its own config, and a pure synonym substitution flipped one verdict from 99.9% to 0.1% confidence. **Rejected.**
2. **Checkpoint #2** (retrain on `theArijitDas/Fake-Reviews-Dataset` , 97% held-out accuracy): failed the *exact same* paraphrase-stability test the *same way* — proof the problem was structural, not one bad checkpoint. **Rejected.**
3. **A candidate replacement dataset** (Amazon "spam/non-spam", 7.57M reviews) was downloaded and inspected — found to be a 100% deterministic proxy for star rating (every 4–5★ review labeled "spam", every 1–3★ labeled "not spam", zero overlap), not a genuine spam judgment at all. **Rejected before any training time was spent on it.**
4. **The dataset actually used:** Ott et al.'s Deceptive Opinion Spam Corpus (Cornell, peer-reviewed) — genuinely human-verified deceptive-vs-truthful labels.
5. **Four training iterations**, each measured against the same held-out split: plain fine-tuning → paraphrase-consistency loss (weight 1.0, then 4.0) → consistency loss extended to cover length-sensitivity too. Each iteration closed one measured gap and, in one case, revealed a *new* one (DistilBERT's length-sensitivity persisted even after two dedicated attempts to fix it).
6. **A parallel TF-IDF + Logistic Regression model** was trained on the same data — found to be inherently more length-robust (no positional/attention mechanism for length to act through) but more prone to small, boundary-adjacent disagreements.
7. **Ensembling the two** (mean probability) combined their complementary strengths.
8. **Statistically meaningful validation** (not 6 hand-picked examples): a WordNet paraphrase was generated for **every one of the 320 held-out test reviews**, with Wilson 95% confidence intervals computed on the resulting flip rate — the ensemble: 0/300 confident flips (95% CI upper bound 1.3%), 6.2% honestly reported as `UNCERTAIN` rather than forced into a guess.

9. **A second, purely engineering constraint:** the DistilBERT component alone is ~257MB — a real OOM risk on Render's free tier (the exact kind of problem from Incident 1). Solved with a `FAKE_REVIEW_TFIDF_ONLY` mode: the ~350KB TF-IDF component alone, independently measured at 0/188 confident flips, at the cost of a higher (41.2%) abstain rate. **This is what the live deployment actually runs.**
10. **Enabled live and verified directly:** a real `POST /pipeline` call against production returned a correctly-labeled `UNCERTAIN` verdict with the right disclaimer text, and `/health` stayed green before and after — confirmed no memory regression.
11. **Published:** model weights + a full model card on Hugging Face (`huggingface.co/RadwaElashry2030/baseera-fake-review-ensemble`), cross-linked from both `README.md` and `MODEL_COMPARISON_AUDIT.md`.

Honest, unresolved limitation carried forward: trained on hotel reviews, applied here to Olist e-commerce reviews — a real domain shift, not separately measurable because no genuinely-labeled Olist fake-review dataset exists.

Database verification

`alembic check` run against a fresh database with every migration applied returned *"No new upgrade operations detected"* — the migrations and `models.py` are provably in sync, not just assumed to be. Schema reviewed against the app's actual needs (no speculative Users/Orders/Auth tables — this app has none of those concepts) and confirmed working on the live database via a real `/predict` call returning a non-null `analysis_id`.

What "actually applied" means in this document

Every fix above was one of: - Re-run against the real backend test suite (129+ tests, all passing after each change) - Verified with a real `curl /API` call against the live deployment - Confirmed via a CI job actually passing (not just "should pass") - Measured with a statistically meaningful sample and a stated confidence interval, where a claim was about reliability rather than a binary pass/fail

Where a fix did not fully close the gap (DistilBERT's residual length-sensitivity being the clearest example), that is stated as-is rather than rounded up to "fixed."